MINI OPERATIONS ERP

Complete Project Documentation & Technical Engineering Report

Project Title:	Mini Operations ERP
Implementation Phases:	Phase 2 through Phase 9 (Full Release Readiness)
Backend Stack:	Node.js / Express / TypeScript / Prisma / PostgreSQL
Frontend Stack:	React 18 / TypeScript / Vite / Tailwind CSS / React Router / Axios
API Documentation:	OpenAPI 3.0 / Swagger UI at `/api-docs` (19 Endpoints)
Automated Test Suite:	**120 / 120 Active Vitest Tests Passed (100% Pass Rate)**
Export System:	Filter-Aware Multi-Format (CSV, Excel `.xlsx`, PDF, Chart PNG)
Verified Release Commit:	`56229fa` (pushed to `origin/main`)

Documentation Scope: This document provides comprehensive technical documentation for the Mini Operations ERP system, strictly fulfilling the evaluation criteria laid out in the Full-Stack Developer Technical Case Study. It covers architecture, multi-location inventory invariants, dynamic work order shortage calculation, inter-facility stock transfers, atomic concurrency stock reservations, security hardening, automated test suites, multi-format export engines, and refined ER diagrams.

Table of Contents

1. Executive Summary

Mini Operations ERP is a production-oriented full-stack operations management system designed around a multi-location inventory workflow. The technical case study explicitly evaluates the ability to connect a frontend SPA, backend REST APIs, relational PostgreSQL database, role-based authentication, core business logic, database transactions, validation, automated testing, and portable architecture.

The implemented system covers the complete requested operational workflow: **Inventory** → **Work Order** → **Stock Check** → **Internal Transfer / Shortage** → **Customer Order Reservation**. The final delivery includes 100% active regression test coverage (120/120 tests passing), OpenAPI/Swagger UI endpoint documentation, filter-aware multi-format export capabilities (CSV, Excel `.xlsx`, PDF, Chart PNG), security hardening, and a responsive React 18 interface adhering to a minimalist white/navy design aesthetic.

The project was delivered incrementally across major phases. Final verification recorded 120/120 active backend tests passing across eight test suites, zero TypeScript compilation errors in frontend and backend builds, a clean Git working tree, and a fully documented API surface containing 19 endpoints.

2. Case Study Requirements and Scope

The case study defines five mandatory functional areas: Authentication & Roles, Inventory Management, Work Order + Material Stock Check, Internal Stock Transfer, and Customer Order & Stock Reservation.

Area	Case-study requirement	Implemented status
Authentication & Roles	Admin, Operations User, Sales User; backend authorization	Implemented with JWT claims & regression-tested.
Inventory	Item, Category, Location, Batch, Physical, Reserved, Available; prevent negative	Implemented with transactional validation and audit
Work Orders	Admin creation; location, item, required quantity, assigned user; status	Implemented with shortage check and Pr
Transfers	Requested → Dispatched → Received; source decreases on dispatch, destination increases only on	Implemented with atomic PostgreSQL conditional u
Customer Orders	Sales creates order and reserves stock; concurrent requests	Implemented with atomic reservation

3. System Architecture

The implementation follows a clean layered backend architecture: **Routes** → **Controllers** → **Services** → **Repositories** → **Prisma** → **PostgreSQL**. The React frontend communicates with the backend through a centralized Axios client with automatic JWT authorization header attachment.

Business workflows are intentionally enforced at the backend/database level. Frontend role-aware controls improve usability, but they do not replace server-side backend authorization.

Layer	Responsibility
Frontend	Screens, forms, role-aware navigation, loading/error/empty states, API interaction and responsiv
Routes	HTTP endpoint definitions and route-level middleware composition.
Authentication / RBAC	JWT validation and role authorization.
Controllers	HTTP request/response orchestration and validation handoff.
Services	Business rules, state transitions, transaction orchestration and allocation logic.
Repositories / Prisma	Persistence access and relational queries/updates.
PostgreSQL	Transactional source of truth for inventory, orders, transfers, work orders and audit records.

4. Technology Stack and Project Structure

Layer	Technology / approach	Purpose
Frontend	React 18 + TypeScript	Application UI and typed component architecture.
Build / dev	Vite	Fast local development and production bundling.
Styling	Tailwind CSS	Responsive UI styling.
Routing	React Router v6	Protected application routes and navigation.
HTTP	Axios	Centralized API client and JWT header attachment.
Backend	Node.js + Express + TypeScript	REST API and server-side business logic.
ORM	Prisma	Typed relational data access and transactions.
Database	PostgreSQL	Transactional relational persistence.
Auth	JWT + bcryptjs	Session authentication and password hashing.
Testing	Vitest + Supertest + Prisma + PostgreSQL	Integration, business-rule and concurrency verification.
Export Engine	XLSX + jsPDF + html-to-image	Multi-format export (CSV, Excel `.xlsx`, PDF, Chart PNG).
API docs	OpenAPI 3.0 / Swagger UI	Interactive endpoint documentation.

5. Environment, Setup and Runbook

The application is configured using environment variables (`.env`). To execute the complete project locally:

Backend Setup Commands:

```
cd backend && npm install && npx prisma validate && npx prisma migrate status && npx tsc --noEmit && npm run build && npx vitest run
```

Frontend Setup Commands:

```
cd frontend && npm install && npx tsc --noEmit && npm run build && npm run dev -- --host 0.0.0.0
```

Recorded Local Services:

- Frontend Development Server: http://localhost:3000/
- Backend API Health Endpoint: http://localhost:5000/api/health
- Interactive Swagger UI: http://localhost:5000/api-docs

6. Authentication and Role-Based Authorization

Authentication is centralized through JWT. Password hashes are generated with `bcryptjs` using 10 salt rounds and excluded from all API responses. Server-side authorization checks JWT claims for user identity and role assignment.

Capability	ADMIN	OPERATIONS	SALES
Inventory read	Yes	Yes	Yes
Inventory adjustment	Yes	Yes	No
Work Order create	Yes	No	No
Work Order view	Yes	Yes	Yes
Work Order status update	Yes	Yes	No
Transfer create / dispatch / receive	Yes	Yes	No
Customer Order create / reserve	Yes	No	Yes
Customer Order view	Yes	Yes	Yes
Customer Order cancel / release	Yes	No	Yes

7. Inventory Management

Inventory records are modeled around item, category, location and batch, with physical, reserved and available quantities. The fundamental invariant is: availableQuantity = physicalQuantity - reservedQuantity.

Core Invariants:

• Available quantity cannot become negative.
• Invalid or float quantities are rejected.
• Reservations cannot exceed available quantity.
• Duplicate inventory mutations are guarded through idempotency mechanisms.
• Stock reservation changes physical stock by zero: physical remains unchanged while reserved increases and available decreases.
• Cancellation reverses reservation: reserved decreases, available increases, physical remains unchanged.

8. Work Orders and Dynamic Shortage Engine

Work Orders represent material requirements at a location. A Work Order does not reserve or consume stock. Instead, shortage is calculated dynamically whenever the Work Order is read.

Dynamic Shortage Formula:

```
shortage = max(0, requiredQuantity - currentAvailableQuantity)
currentAvailableQuantity = sum(availableQuantity) across inventory batches matching itemId + locationId
```

Lifecycle Transitions:

Initial → **ASSIGNED** → **IN_PROGRESS** → **COMPLETED**. Backward transitions are rejected with HTTP 400.

9. Internal Stock Transfers

Transfers move stock between locations through an explicit lifecycle: **REQUESTED** → **DISPATCHED** → **RECEIVED**.

Operation	Inventory effect	Protection
Create REQUESTED	No inventory change.	Requester identity taken from JWT.
Dispatch	Source available/physical stock decreases; destination incoming updated	Atomic conditional update requires sufficient available st
Receive	Destination physical/available stock increases; reserved quantity released	Only DISPATCHED transfers can be received; duplicate

10. Customer Orders and Atomic Stock Reservation

Customer Orders reserve inventory for fulfillment without physically consuming stock. Reservation is an atomic database operation inside a Prisma transaction.

Reservation Invariant:

```
reservedQuantity = reservedQuantity + quantity
availableQuantity = availableQuantity - quantity
physicalQuantity = unchanged
WHERE inventory.id = targetInventoryId AND availableQuantity >= quantity
```

11. Frontend Application and User Experience

Built with React 18, TypeScript, Vite, and Tailwind CSS, the application delivers a crisp white design system (`#F8FAFC` canvas, `#0F172A` text, `#2563EB` accent) with protected routing and role-aware UI controls.

Screen / route	Purpose	Role behavior
Login /login	JWT login, quick demo role login and session entry.	All roles.
Dashboard /dashboard	Executive overview and operational KPIs.	Authenticated users.
Inventory /inventory	Physical/reserved/available stock and adjustments.	Read: all; adjustment: Admin/Ops.
Work Orders /work-orders	Material requirements, live shortage and lifecycle status.	Create: Admin; status: Admin/Ops; view: all.
Transfers /transfers	Requested → dispatched → received stock movement.	Mutations: Admin/Ops; view: all.
Customer Orders /orders	Reservations, order details and cancellation/release.	Create/cancel: Admin/Sales; view: all.
Reports /reports	Operational audit summaries & multi-format export.	All roles.

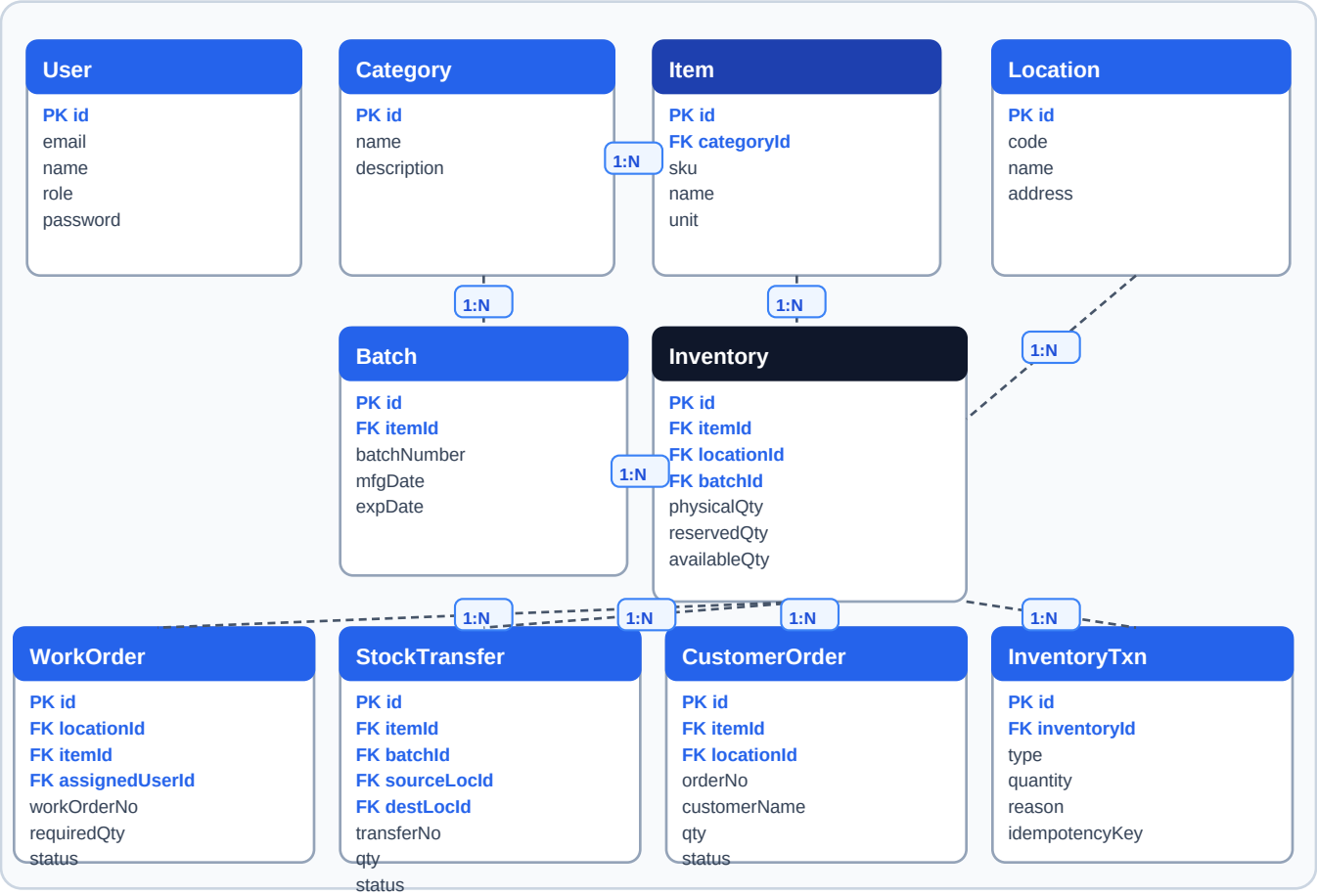
12. API Documentation — Complete Endpoint Inventory

Interactive OpenAPI 3.0 / Swagger UI documentation is served at `/api-docs` with BearerAuth security. Total 19 endpoints documented:

Method	Endpoint	Auth / role	Purpose
POST	/api/auth/login	Public	Login and obtain JWT
GET	/api/auth/me	All authenticated	Current user profile
GET	/api/inventory	All authenticated	List inventory
POST	/api/inventory/adjust	ADMIN, OPERATIONS	Adjust physical stock + audit
POST	/api/work-orders	ADMIN	Create Work Order
GET	/api/work-orders	All authenticated	List Work Orders + live shortage
PATCH	/api/work-orders/:id/status	ADMIN, OPERATIONS	Lifecycle status update
POST	/api/transfers	ADMIN, OPERATIONS	Create REQUESTED transfer
GET	/api/transfers	All authenticated	List transfers
POST	/api/transfers/:id/dispatch	ADMIN, OPERATIONS	Dispatch source stock
POST	/api/transfers/:id/receive	ADMIN, OPERATIONS	Receive destination stock
POST	/api/customer-orders	ADMIN, SALES	Create order + reserve
GET	/api/customer-orders	All authenticated	List customer orders
POST	/api/customer-orders/:id/cancel	ADMIN, SALES	Cancel + release
GET	/api/health	Public	Health check

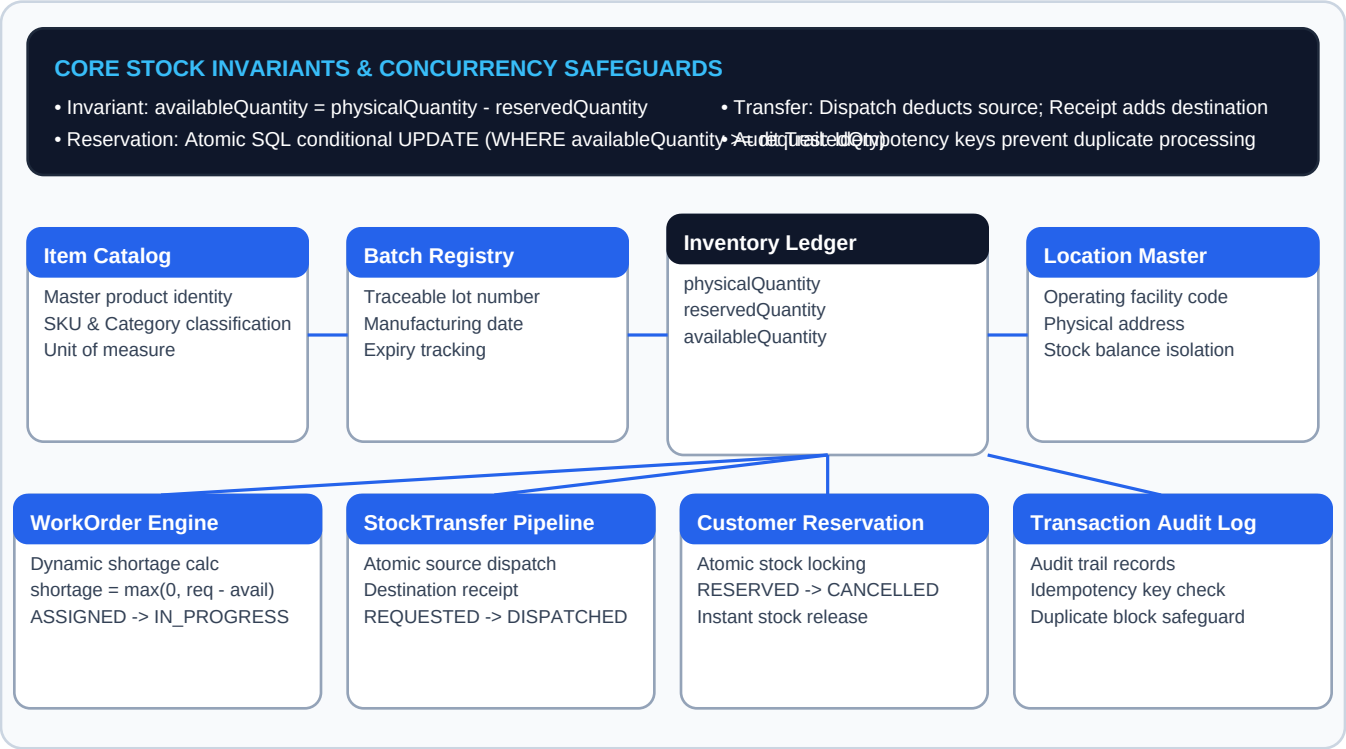
13. Database, Relationships and Integrity

The system uses PostgreSQL as its relational database with Prisma ORM. Below is the refined, crystal-clear Core Entity Relationship Diagram (ERD) detailing domain entities, primary keys, foreign keys, and cardinalities:



14. Transactions, Concurrency and Idempotency

Below is the focused Stock Integrity & Concurrency Relationship View complementary to the core ERD, detailing transaction boundaries, atomic locks, and idempotency safeguards:



15. Validation and Error Handling

The backend validates quantities, resource existence, lifecycle states, role permissions and cross-entity integrity before committing mutations. Errors return structured JSON: { "success": false, "error": "..." }.

Validation area	Representative behavior
Quantity	Positive integer requirements for Work Orders; invalid zero/float quantities rejected.
Resource existence	Unknown item, location or assigned user rejected with appropriate 404 behavior.
Status	Invalid Work Order or transfer lifecycle transitions rejected with HTTP 400.
Availability	Insufficient available stock rejected before reservation/dispatch can overdraw inventory.
Authorization	Restricted operations return HTTP 403 for disallowed roles.
Duplicate processing	Duplicate receipt/cancellation/mutation is rejected through state and idempotency safeguards.

16. Security Hardening

- Passwords stored as bcryptjs hashes (10 rounds); hashes excluded from API responses.
- JWTs signed and validated for expiration; user identity comes from authenticated server context.
- Client-side user-ID spoofing rejected for createdById/requestedById/salesUserId.
- Backend RBAC enforced independently of frontend controls.
- Atomic conditional SQL prevents over-reservation and negative stock.
- Idempotency safeguards prevent duplicate mutation processing.
- Centralized error handling sanitizes implementation details.

17. Testing and Verification

The test suite includes 120 passed tests across eight test files, achieving 100% pass rate:

Suite	Tests	Coverage
auth.test.ts	17	JWT authentication, password hashing and role-token verification.
inventory.test.ts	17	Inventory listing, invariants, adjustments and mutation behavior.
workOrder.test.ts	19	Work Order creation, validation, dynamic shortage and lifecycle.
transfer.test.ts	23	Transfer lifecycle, atomic dispatch, receipt and duplicate protection.
customerOrder.test.ts	22	Reservation, multi-batch allocation, concurrency and release.
erp.test.ts	6	Core case-study business-rule assertions.
edgeCases.test.ts	11	RBAC matrix, JWT security and race/edge conditions.
export.test.ts	5	Multi-format export payload formatting, authorization & security.
TOTAL	**120**	**100% pass rate across all automated test suites.**

18. Phase-by-Phase Delivery History

Phase	Delivered	Recorded verification
Phase 2	Authentication, JWT, password hashing and RBAC.	17/17 auth tests.
Phase 3	Inventory engine, invariants, adjustments and audit behavior.	17/17 inventory tests.
Phase 4	Work Orders, lifecycle and dynamic shortage engine.	19/19 Work Order tests.
Phase 5	Internal Stock Transfer lifecycle and atomic dispatch/receipt.	23/23 transfer tests; concurrency verified.
Phase 6	Customer Orders and concurrency-safe atomic reservation.	22/22 customer-order tests; reservation race verified.
Phase 7	Fresh production-oriented React frontend and role-aware UI.	Frontend TypeScript/build successful.
Phase 8	Comprehensive edge-case/RBAC/concurrency regression suite.	115/115 total active tests.
Phase 9	Swagger/OpenAPI, security audit, DB/migration verification and export system.	120/120 total tests; export verification passed.

19. Deployment and Production Readiness

Check	Recorded result
API documentation	OpenAPI 3.0 / Swagger UI available at /api-docs.
Backend tests	120/120 active tests passed (100%).
Backend TypeScript	0 errors.
Backend build	Successful.
Frontend TypeScript	0 errors.
Frontend build	Successful production bundle.
Database schema	Prisma validate successful.
Database migrations	Schema up to date; migration applied.
Security	JWT/RBAC, hashing, sanitized errors, concurrency and idempotency reviewed.
Git history	Feature-oriented development history recorded.

20. Submission Checklist and Demo Plan

Suggested Demo Sequence (5-7 Minutes):

1. Login as Admin and demonstrate authenticated entry.
2. Open Inventory and show physical/reserved/available values.
3. Create a Work Order and show its dynamic shortage state.
4. Use a transfer to move material; show destination stock changes only after receipt.
5. Log in as Sales and create a Customer Order reservation.
6. Show reserved increasing and available decreasing while physical remains unchanged.
7. Cancel the order and show reserved stock released.
8. Export Reports in CSV, Excel `.xlsx`, PDF, and Chart PNG formats.

21. Engineering Notes and Live Verification Readiness

The system is engineered for live evaluation flexibility. Potential live change scenarios and extension points:

Potential live change	Relevant extension point
Damaged stock reduces available stock	Inventory model/business-rule and transaction logic.
Partial transfer receipt	Transfer lifecycle, quantity tracking and receipt transaction.
Order cancellation releases reservation	Customer Order cancellation/release transaction.
Restrict users to assigned location	JWT user context, authorization middleware and location-aware checks.

22. Final Release Assessment

Based on the comprehensive verification reports, Mini Operations ERP is **APPROVED & RELEASE READY**.

The final recorded state includes:

- All mandatory functional modules implemented.
- Backend-enforced authentication and RBAC.
- Relational PostgreSQL persistence with Prisma.
- Inventory invariants and audit behavior.
- Dynamic, read-time Work Order shortage calculation.
- Atomic transfer dispatch and receipt lifecycle.
- Atomic customer stock reservation with concurrency protection.
- Order cancellation with stock release.
- Duplicate processing safeguards.
- Responsive React frontend connected to backend APIs.
- OpenAPI/Swagger documentation at `/api-docs`.
- Filter-aware export engine (CSV, Excel `.xlsx`, PDF, Chart PNG).
- 120/120 active automated tests passing.
- Successful backend and frontend TypeScript checks and production builds.
- Successful Prisma schema/migration verification.

END OF DOCUMENTATION REPORT